

函数

直觉:

- 函数大概有什么用?
 - 参数 --> 函数体里面的指令 --> 返回
 - 把函数看成是程序的组件:
 - 可以把函数看成是小的子程序
 - 可以程序看成是不同的函数组合起来
- 使用函数有什么好处?
 - 减少重复代码的冗余

例子

现在有三组，每组里面有3个整数，请问每组里面的数据能否作为三角形的边长？

1, 2, 3

2, 3, 4

4, 5, 6

任意两边之和要大于第三边

不使用函数的话，代码有很多冗余

```
int main() {  
    int a1 = 1, b1 = 2, c1 = 3;  
    int a2 = 2, b2 = 3, c2 = 4;  
    int a3 = 4, b3 = 5, c3 = 6;  
    if (a1 + b1 > c1 && a1 + c1 > b1 && b1 + c1 > a1) {  
        printf("Is triangle!\n");  
    }  
    else {  
        printf("Not triangle!\n");  
    }  
    if (a2 + b2 > c2 && a2 + c2 > b2 && b2 + c2 > a2) {  
        printf("Is triangle!\n");  
    }  
    else {  
        printf("Not triangle!\n");  
    }  
    if (a3 + b3 > c3 && a3 + c3 > b3 && b3 + c3 > a3) {  
        printf("Is triangle!\n");  
    }  
    else {  
        printf("Not triangle!\n");  
    }  
    return 0;  
}
```

使用函数以后代码很精简

```
// 暂时先把函数写在main函数的前面
// 返回值类型 函数名字 函数参数 函数体
void is_triangle(int a, int b, int c) {
    if (a + b > c && a + c > b && b + c > a) {
        printf("Is triangle!\n");
    }
    else {
        printf("Not triangle!\n");
    }
}

int main() {
    int a1 = 1, b1 = 2, c1 = 3;
    int a2 = 2, b2 = 3, c2 = 4;
    int a3 = 4, b3 = 5, c3 = 6;
    is_triangle(a1, b1, c1);
    is_triangle(a2, b2, c2);
    is_triangle(a3, b3, c3);
}
```

函数的声明

通知编译器函数的一些信息： 返回值类型、函数名、形式参数

在使用函数之前，必须有函数的声明

```
void is_triangle(int a, int b, int c); //函数的声明
void is_triangle(int a, int b, int c); //声明可以多次
int main() {
    int a1 = 1, b1 = 2, c1 = 3;
    int a2 = 2, b2 = 3, c2 = 4;
    int a3 = 4, b3 = 5, c3 = 6;
    is_triangle(a1, b1, c1);
    is_triangle(a2, b2, c2);
    is_triangle(a3, b3, c3);
}
```

声明可以多次

函数的定义

函数的定义就是把一个函数的所有信息全部描述出来

返回值类型 函数名字 形参列表 函数体

```
void is_triangle(int a, int b, int c) {  
    if (a + b > c && a + c > b && b + c > a) {  
        printf("Is triangle!\n");  
    }  
    else {  
        printf("Not triangle!\n");  
    }  
}
```

函数定义会自带一次声明

函数定义在一个程序当中，只能有一次

函数调用

函数调用语句之前，必须要先有函数声明

```
is_triangle(a1, b1, c1);  
is_triangle(a2, b2, c2);  
is_triangle(a3, b3, c3);
```

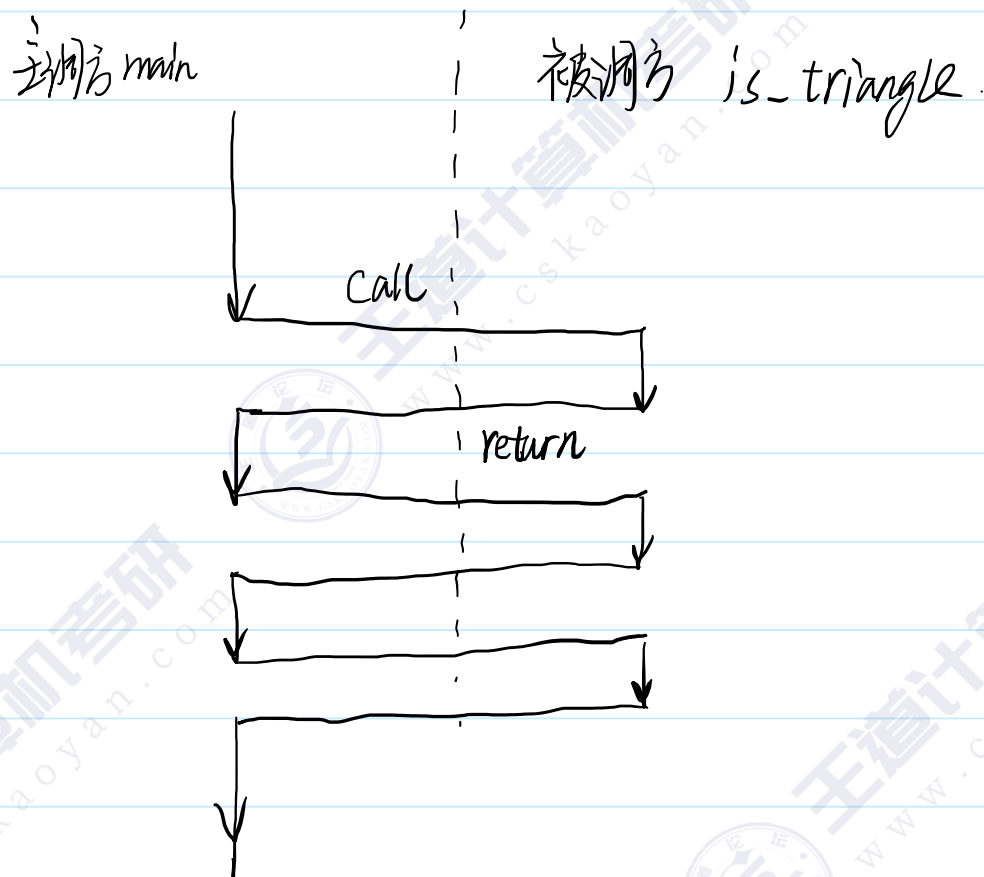
调用函数时，先写函数的名字，再写一对圆括号，在圆括号里面写传入的参数（实际参数实参）

主调方：调用函数的这一方

被调方：被调用的函数

流程：主调方先准备实参，然后传入给被调方，调用被调函数，被调方执行函数体，返回之后将返回值给主调方

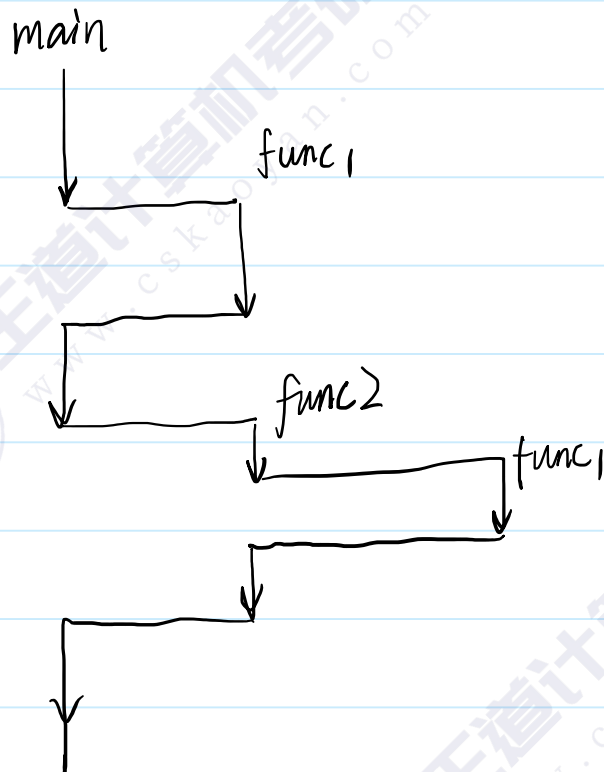
调试函数调用过程



- 1、调试函数的时候要使用F11而不是F10
- 2、主调函数去调用被调函数的时候，指令位置会跳转到被调函数，等被调函数返回之后，再回到主调方

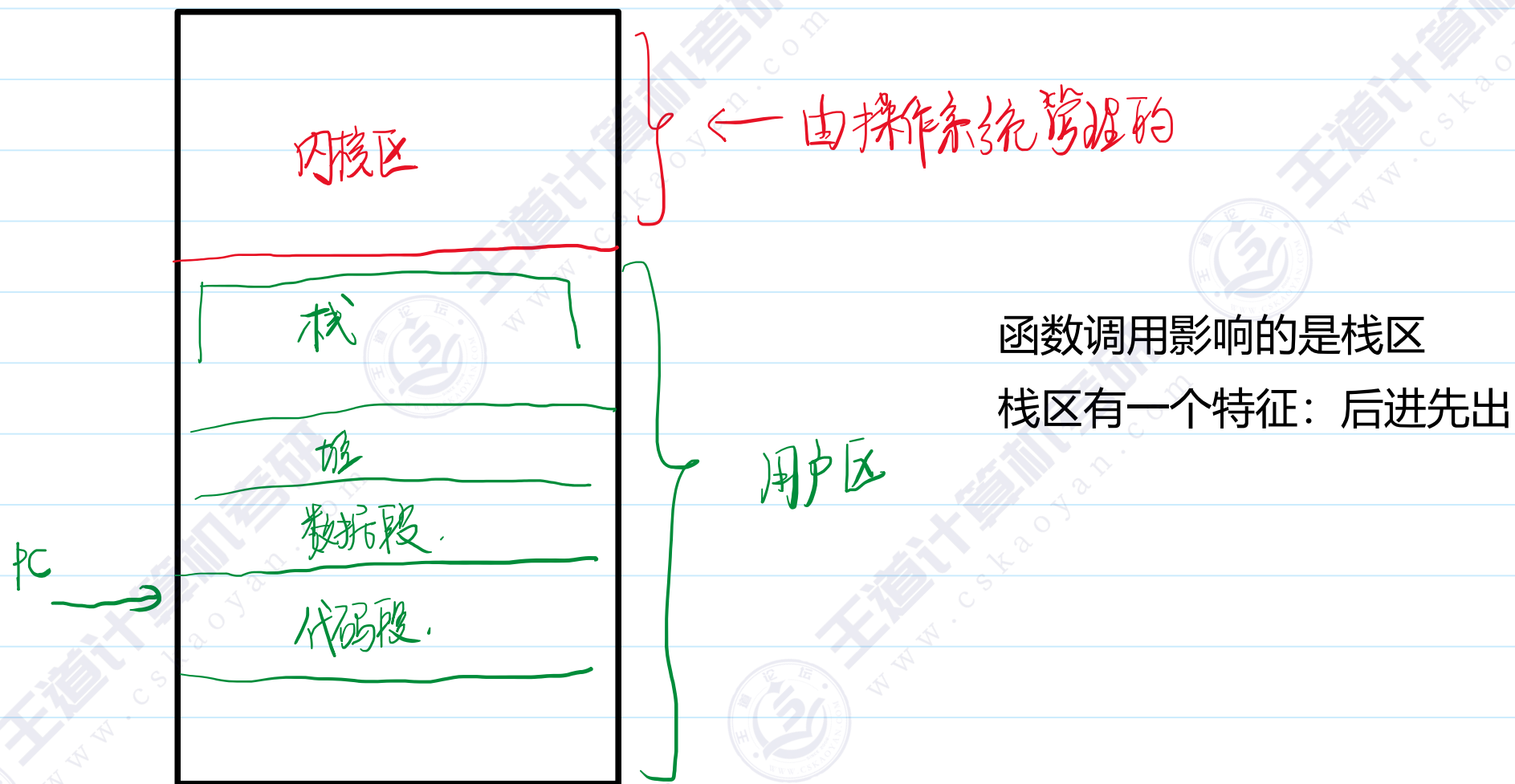
```
void func1() {  
}  
void func2() {  
    func1();  
}  
int main() {  
    func1();  
    func2();  
    return 0;  
}
```

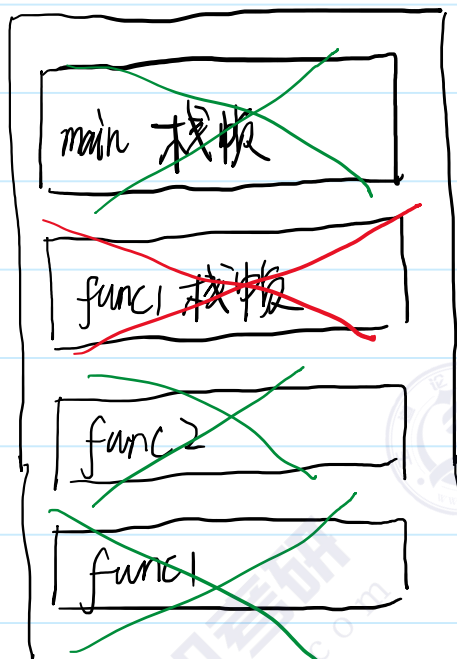
已用时间 <= 1ms



后调用的函数先返回

函数调用的内存原理





每次调用函数，会开辟一个栈帧

栈帧会压入栈区；

每次函数返回，会弹出一个栈帧

栈区的特征是后进先出

作用域

描述变量的有效范围

```
int a = 1;
void func() {
    int b = 2;
}
int main() {
    int c = 3;
    {
        int d = 4;
        // a b c d
    }
    // a b c d
    return 0;
}
```

变量的作用域是以{}作为边界的

外部变量在内部依然可以生效

内部变量在外部不生效

C语言不支持函数嵌套定义：

- 函数外部的叫做全局
- 函数内部的叫做局部

```
int a = 1;
void func() {
    int b = 2;
}
int main() {
    int c = 3;
    {
        int d = 4;
        int a = 2;
        a = 3;
        // a b c d --> 除了b
    }
    // a b c d --> 除了b和d
    return 0; 已用时间 <= 1ms
}
```

在内部作用域里面，可以起一个和外部变量重名的变量 ---> 隐藏

当程序运行到内部作用域的时候，外部变量就被藏起来了

全局a被藏起来了。在这个作用域中，只能访问内部a。

生存期

程序运行的时候，变量申请内存和释放内存的时机

作用域，在写代码的时候考虑的

生存期，是在程序运行的时候考虑的

示意图

```
int a = 1;  
void func() {  
    int a = 2;  
}  
int main() {  
    int a = 3;  
    func();  
    return 0;  
}
```

程序一开始就有效。

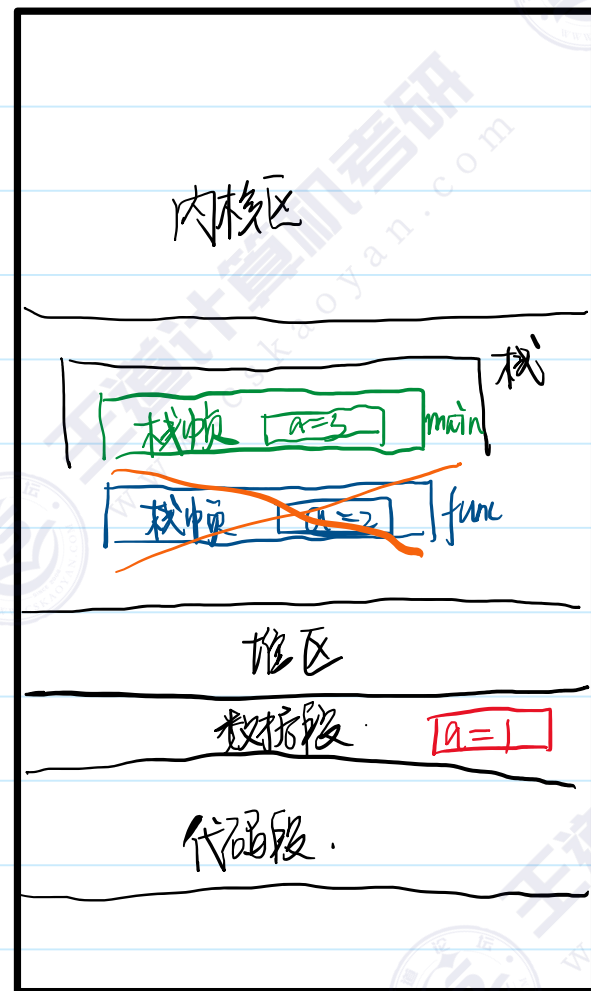
程序结束失效。

PC →

局部变量在栈帧里面。

函数调用时分配内存

函数返回时释放内存。



值传递

形参 等价于 swap 的局部变量。

```
void swap(int a, int b) {  
    int temp = a;  
    a = b;  
    b = temp;  
    printf("swap, a = %d, b = %d\n", a, b);  
}  
int main() {  
    int a = 10, b = 5;  
    swap(a, b);  
    printf("main, a = %d, b = %d\n", a, b);  
    return 0;  
}
```

实参 用实参值初始化形参，

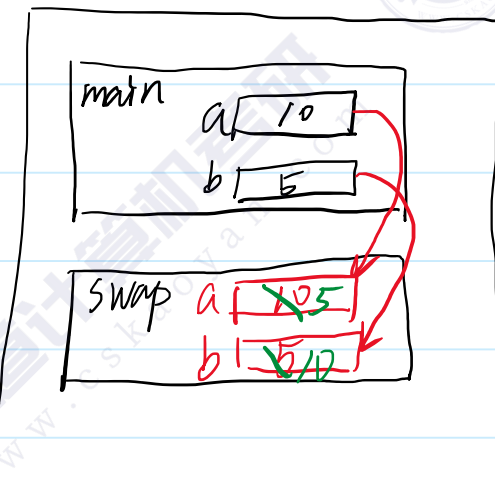
10 5

Microsoft Visual Studio 调试控制台

```
swap, a = 5, b = 10  
main, a = 10, b = 5
```

在swap函数里面，交换成功
在main函数里面，交换失败

值传递。



数组

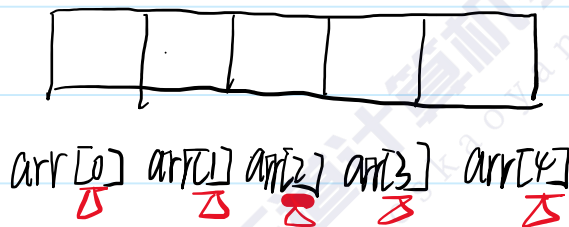
场景:

- 如果需要5个变量? `int a1,a2,a3,a4,a5;`
- 如果需要10个变量? `int a1, ... a10;`
- 如果需要100个变量?

聚合数据结构 --> 数组

- 连续
- 元素的数据类型一致
- 元素的下标是有序的

`int arr[5];`



下标/索引. 从0开始计数.

数组的定义

元素的数据类型 数组名[数组长度];

正常数
不能是变量
>0

```
#define LEN 5
int main() {
    //数组的长度必须大于0&&是一个整数
    int arr[LEN]; //数组元素类型是int 长度为5 数组名是arr
    //下面是不推荐的写法 数组长度不应该是变量
    // int len = 5;
    // int arr[len];
    return 0;
}
```

数组的初始化

就是在定义的时候，就给数组的元素设置好值

访问数组元素

```
int arr[5] = {1,2,3,4,5}
```

```
// 下标运算符 []
```

```
arr[0] arr[1] ... arr[4] // 范围是0 ~ N-1
```

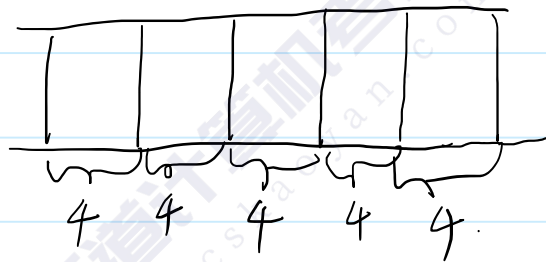
→ 一个元素. 可以当成一个变量. { ① 里面有值
② 有内存空间

数组元素的内存布局

`int arr[5]`

① 连续.

② 类型相同.



0x0000007F87F0F578	01 00 00 00
0x0000007F87F0F57C	02 00 00 00
0x0000007F87F0F580	03 00 00 00
0x0000007F87F0F584	04 00 00 00
0x0000007F87F0F588	05 00 00 00

1、数组的首地址和`arr[0]`的首地址是一样的

2、`arr[i]`的地址 = 数组首地址 + $i * \text{sizeof}(\text{元素类型})$ ==>

如果想要访问某个元素，不需要知道数组的长度

3、`[]`运算符的本质：是先计算地址、再访问元素

数组越界的例子

```
int arr[5] = { 1,2,3,4,5 };  
int a = 20;  
for (int i = 0; i < 13; ++i) {  
    arr[i] = i + 1;  
}  
printf("a = %d\n", a);
```

C:\Users\liaozi\source\repos\C语言督学营\x64\Debug\017_数组.exe

a = 12

已引发异常

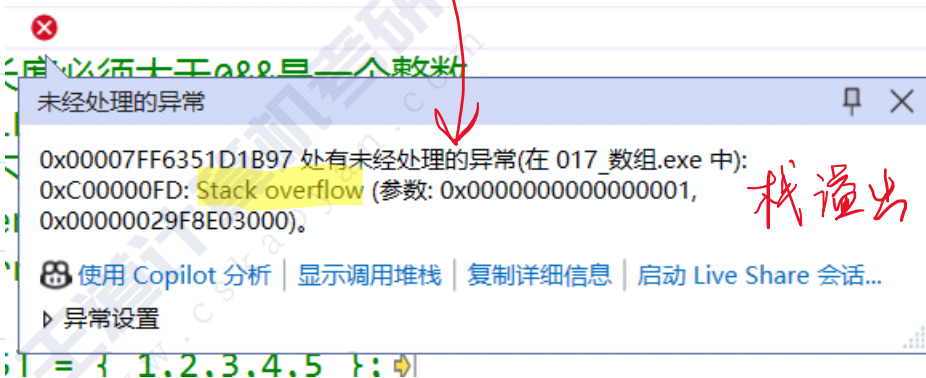
Run-Time Check Failure #2 - Stack around the variable 'arr' was corrupted.

 使用 Copilot 分析 | [显示调用堆栈](#) | [复制详细信息](#) | [启动 Live Share 会话...](#)

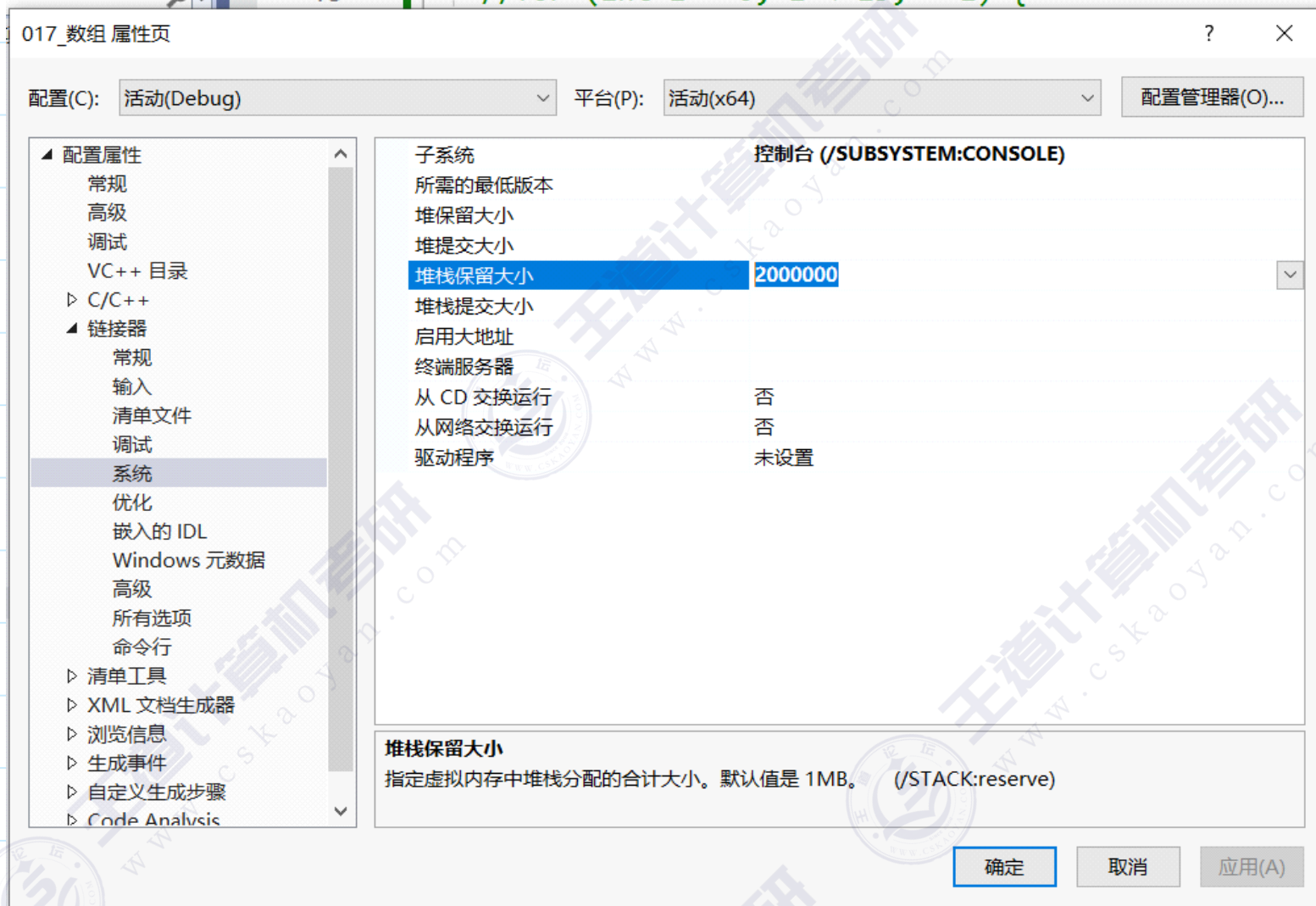
栈帧的大小限制

只要是局部的变量，无论是单一变量还是数组，都分配在栈帧上面，数组的长度有没有限制？

```
//char arr[1000000];  
char arr[1200000];
```



修改栈帧的大小限制

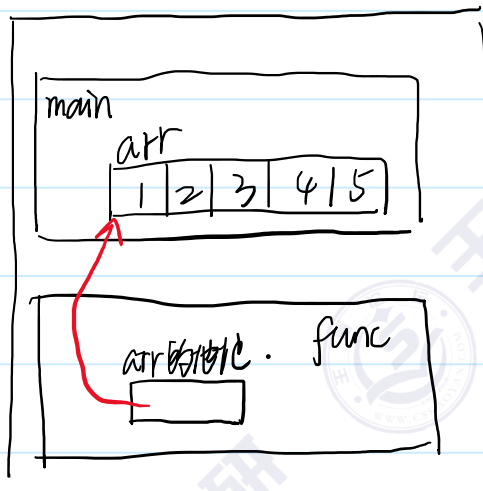


使用函数传递数组

[]运算符

arr[i]的原理 先根据数组首地址、元素类型和下标i去计算一个地址，再根据地址去访问内存 --> 数组的长度没用的信息

C语言的作者在设计数组参数的时候，决定丢弃掉长度信息



```
#include <stdio.h>
```

```
#define LEN 5
```

```
void func(int *arr, int length) { //length补充的参数
```

```
    //在被调函数中，数组会退化成一个地址，丢失了长度信息
```

```
    printf("func sizeof(arr) = %d\n", sizeof(arr));
```

```
    for (int i = 0; i < length; ++i) {
```

```
        printf("%3d", arr[i]); //[]运算符不需要长度信息
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int main() {
```

```
    //在主调函数的位置，我们是知道数组的长度信息
```

```
    int arr[LEN] = { 1, 2, 3, 4, 5 };
```

```
    printf("main sizeof(arr) = %d\n", sizeof(arr));
```

```
    for (int i = 0; i < sizeof(arr)/sizeof(int); ++i) {
```

```
        printf("%3d", arr[i]);
```

```
    }
```

```
    printf("\n");
```

```
    func(arr, sizeof(arr)/sizeof(int)); //数组这个整体作为实参时，不需要[]
```

```
    return 0;
```

```
}
```

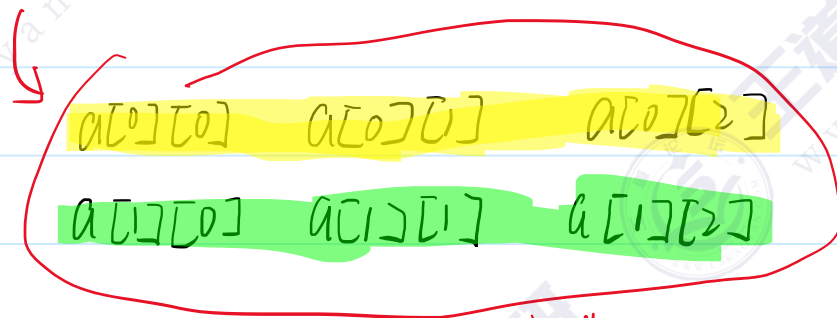
二维数组

一维数组 ---> 向量

二维数组 ---> 矩阵

`int arr[M][N]` M行数 N列数

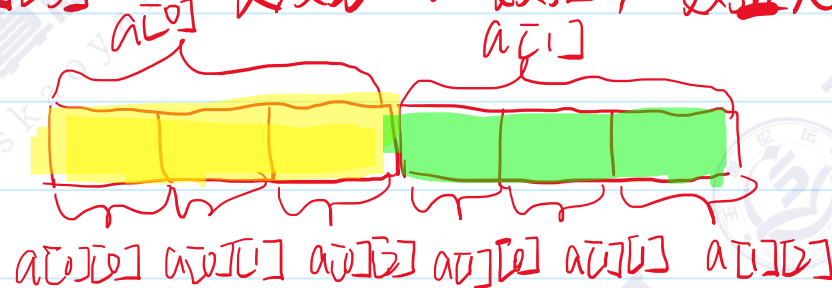
`int a[2][3]`



从计算机角度来看，二维数组也是一维数组

- 元素类型是数组 —> "数组的数组"
- 长度是行数

`int a[2][3]` 长度为2的数组，数组元素也是长度为3的数组



→ 按照行优先存储

`int arr[2][3]`

arr[i][j]的地址如何计算?

先找arr[i][0], 再偏移

arr首地址 + i * sizeof(int) * 3 + j * sizeof(int)

行数不需要, 列数需要.